

AI+金融财报分析系统 — 仓库目录结构说明

本文档详细标注了仓库中每个文件夹和文件的具体用途、设计理由与边界约定。
架构基础请参见 [架构-v4.1.md](#)

根目录

```
ai-finance-analyzer/  
├── README.md  
├── pyproject.toml  
├── .env.example  
├── .gitignore  
├── config/  
├── schemas/  
├── pipeline/  
├── layers/  
├── llm/  
├── data/  
├── tests/  
├── notebooks/  
└── scripts/
```

README.md — 项目总入口

用途：项目首页，新成员第一眼看到的东西。

应包含：

- 一句话：这个项目做什么（AI 解读上市公司年报）
- 架构图（六层 + 隔离原则）
- 快速开始： `pip install -e .` → `python scripts/run_pipeline.py --pdf xxx.pdf`
- 目录结构速览（指向本文档）
- 依赖要求（Python 版本、API Key 配置方式）
- 贡献指南 / 团队约定

pyproject.toml — 项目元信息与依赖管理

用途：PEP 621 标准项目配置，替代传统的 `setup.py` + `requirements.txt`。

包含：

- `[project]`：名称、版本、作者、Python 版本要求 (≥ 3.11)
- `[project.dependencies]`：运行时依赖
 - `pydantic`（schemas 层类型定义）
 - `pandas`、`numpy`（B1 / B+ / C 层计算）
 - `openai` / `anthropic`（LLM 调用）
 - `pyyaml`（加载 config/ 下的 yaml）
- `[project.optional-dependencies]`：
 - `dev`：pytest, ruff, mypy
 - `pdf`：pdfplumber, MinerU（第 0 层）
- `[tool.ruff]`：代码风格配置
- `[tool.pytest.ini_options]`：测试配置

边界约定：

- 生产依赖仅包含**运行时需要**的库
- 分析和调试工具放在 `optional-dependencies` 的 `dev` 组

`.env.example` — 环境变量模板

用途：告诉开发者需要配置哪些环境变量，避免把真实 API Key 提交到 git。

内容示例：

```
# LLM API 配置
ANTHROPIC_API_KEY=sk-ant-xxxx
OPENAI_API_KEY=sk-xxxx

# 可选：搜索 API（A0 层用）
SERPAPI_KEY=xxxx

# 数据路径（默认即可）
DATA_DIR=./data
```

边界约定：

- 永远不包含真实值
- 真实 `.env` 文件加入 `.gitignore`

`.gitignore` — Git 忽略规则

用途：防止敏感信息、大文件、缓存进入版本控制。

应包含：

```
# 环境变量
.env

# API Key 相关
*.key

# 数据文件（PDF 和输出）
data/raw/*
data/processed/*
data/outputs/*
!data/benchmarks/*.csv # 基准数据是代码生成的，可以跟踪

# Python 缓存
__pycache__/*
*.pyc
.venv/
venv/

# 编辑器
.vscode/
.idea/

# 操作系统
Thumbs.db
.DS_Store

# 大文件
*.pdf
*.json # 可能含财报数据；小样本测试数据用 !data/test_*.json 例外
```

config/ — 全局配置（不变的数据）

```
config/
├─ weights.yaml # E层：各指标的基础影响权重
├─ industry_tags.yaml # A层：行业标签体系定义
├─ thresholds.yaml # B+/C层：异常判定阈值
├─ financial_fields.yaml # B0层：字段统一映射表
├─ prompts/ # 所有 LLM prompt 模板
│   └─ a0_macro_search.yaml
│   └─ a1_tagging.yaml
│   └─ ...
```

config/weights.yaml — 指标影响权重库

用途：E1 打分系统的核心。定义每个财务指标的基础影响权重，以及权重的"方向"（利好/利空/中性）。

字段约定：

```
# 格式
weights:
  存货周转率:
    category: "营运能力"           # 所属类别
    base_weight: -1                 # 基础影响：正数=利好，负数=利空
    direction: "negative"           # 异常方向（偏离大=坏事）
    amplification_cap: 3.0          # MAD 放大系数上限，防止极端值过度扣分
  毛利率:
    category: "盈利能力"
    base_weight: 1
    direction: "positive"           # 异常方向（偏离大=好事）
```

边界约定：

- 只定义权重的"骨架"，不涉及具体数值（具体数值在代码中动态计算）
- 权重值和方向由领域专家审核，非开发人员也可修改

config/industry_tags.yaml — 行业标签体系

用途：定义所有可用的硬标签和软标签体系，A1 层的 LLM 在这里选值，A2 层的匹配算法据此筛选。

```
hard_tags:
- name: "同花顺三级行业"
  description: "同花顺行业分类标准，精确到三级"
  source: "同花顺 iFind"
  # 具体值通过 A1 LLM 动态生成，不在配置文件预定义

soft_tags:
- name: "资产结构"
  options: ["重资产", "轻资产", "混合"]
- name: "毛利率水平"
  options: ["高毛利", "中等毛利", "低毛利"]
- name: "客户类型"
  options: ["To-B", "To-C", "To-G"]
- name: "竞争壁垒"
  options: ["品牌护城河", "技术壁垒", "规模效应", "特许经营", "无明显壁垒"]
- name: "现金流特征"
  options: ["奶牛型", "烧钱型", "平衡型"]
- name: "生命周期"
  options: ["初创期", "成长期", "成熟期", "衰退期"]
```

边界约定：

- 软标签的选项是枚举的，LLM 只能从中选择，不能自创
- 需要新增软标签维度时，团队评审后在此文件添加

config/thresholds.yaml — 异常判定阈值

用途：B+ 层和 C 层的硬编码阈值，决定什么叫做"异常"。

```
bplus:
  净现比:
    normal_range: [0.6, 2.0]      # 正常范围
    severe_threshold: 0.0         # 低于此值视为严重
  存贷双高:
    # 货币资金/总资产 和 有息负债/总资产 都高于此值则可疑
    both_above: 0.3

c_layer:
  mad_threshold: 2.0              # 超过几倍 MAD 视为异常
  extreme_threshold: 5.0         # 超过几倍 MAD 视为极端异常
```

边界约定：

- 阈值是经验值，初期参考学术论文和行业报告设定，后续通过回测优化
- 不同行业可以有不同阈值（扩展时可在 industry 下加子配置）

config/financial_fields.yaml — 财务字段统一映射表

用途：B0 层大模型做字段映射的依据。定义了系统标准字段列表，以及常见的中文变体。

```
fields:
  Revenue_Total:
    chinese_variants:
      - "营业收入"
      - "营业总收入"
      - "主营业务收入"
      - "营收"
    type: "profit_loss"
    unit: "元"
    description: "营业收入（总收入）"

  Cost_Revenue:
    chinese_variants:
      - "营业成本"
      - "主营业务成本"
    type: "profit_loss"
    unit: "元"

# ... 共约 50-80 个标准字段
```

边界约定：

- 这是系统的"共享词汇表", 所有层都引用同一个字段名
- 新增字段时需要在所有层同步更新

config/prompts/ — LLM Prompt 模板库

```
config/prompts/  
├─ a0_macro_search.yaml      # A0: 联网搜索研报的 prompt  
├─ a1_tagging.yaml          # A1: 打行业标签的 prompt  
├─ b0_semantic_guide.yaml    # B0: 识别表头和单位的 prompt  
├─ d1_lookup_notes.yaml      # D1路1: 查原文的 prompt  
├─ d1_hypothesis.yaml        # D1路2: 推演假设的 prompt  
├─ d2_probability.yaml       # D2: 概率分配的 prompt  
└─ e2_report.yaml           # E2: 生成报告文本的 prompt
```

用途: 将所有 LLM prompt 外置为 YAML 文件, 实现"代码与 prompt 分离"。

```
# 示例: a0_macro_search.yaml  
name: a0_macro_search  
description: "搜索行业宏观事实, 屏蔽主观评级"  
  
messages:  
  - role: system  
    content: |  
      你是金融研报分析助手。你的任务是根据给定的搜索词, 从搜索结果中提取**客观事实**。  
  
      规则:  
      1. 只提取数据性内容: 价格涨跌幅、供需数据、政策法规原文  
      2. 严格屏蔽以下内容: 任何"强烈看好/强烈卖出/推荐买入"等主观评级  
      3. 如果搜索结果中没有客观数据, 输出"未找到客观数据"  
      4. 每个事实需标明数据来源和日期  
  
  - role: user  
    content: |  
      搜索关键词: {{search_query}}  
      搜索结果: {{search_results}}
```

边界约定:

- Prompt 使用 Jinja2 模板语法 ({{variable}}), 由 llm/prompt_loader.py 加载渲染
- 每个 prompt 文件有明确的 name 字段, 代码通过 name 引用, 不硬编码路径
- 修改 prompt 不需要修改代码

schemas/ — 跨层共享的数据契约

```
schemas/
├── __init__.py
├── raw_doc.py           # 第0层输出: JSON 四大区块
├── financial.py        # 统一财务字段体系
├── tags.py             # 硬标签 + 软标签 结构
├── anomaly.py          # B+异常清单 / C层MAD偏离度
├── benchmark.py        # A2层: 同行基准数据
├── reasoning.py        # D层: 推理路径与概率
├── report.py           # E层: 最终报告结构
└── enums.py            # 公共枚举 (报表类型、标签类型等)
```

schemas/enums.py — 公共枚举

```
from enum import Enum

class ReportType(str, Enum):
    CONSOLIDATED = "合并报表"
    PARENT = "母公司报表"

class AnomalySeverity(str, Enum):
    LOGIC = "逻辑异常"      # B+ 层
    DEVIATION = "偏差异常"  # C 层

class ConfidenceTier(str, Enum):
    HIGH = "高置信度"
    MEDIUM = "中等置信度"
    LOW = "低置信度"
```

schemas/raw_doc.py — PDF 提取结果

```
from pydantic import BaseModel
from typing import Optional

class FinancialTable(BaseModel):
    """财务数据部分：三大报表的表格原始数据"""
    balance_sheet: list[dict] # 资产负债表 OCR 数据
    income_statement: list[dict] # 利润表 OCR 数据
    cashflow_statement: list[dict] # 现金流量表 OCR 数据

class ManagementDiscussion(BaseModel):
    """管理层讨论与分析部分"""
    sections: list[dict] # 分段文字，含章节标题和内容
    # 每段含: {title, content, page_number}

class Footnotes(BaseModel):
    """附注明细部分"""
    items: list[dict] # 各附注条目
    # 每项含: {name, content, page_number}

class CompanyOverview(BaseModel):
    """公司基本情况部分"""
    business_description: str # 主营业务介绍
    industry_classification: str # 行业分类（原文中的）
    # ...

class RawDocument(BaseModel):
    """第0层的完整输出"""
    metadata: dict # PDF 元数据（文件名、年份、页码数等）
    financial_data: FinancialTable
    management_discussion: ManagementDiscussion
    footnotes: Footnotes
    company_overview: CompanyOverview
```

schemas/financial.py — 统一财务字段

```
from pydantic import BaseModel
from datetime import date

class FinancialField(BaseModel):
    """统一财务字段"""
    standard_name: str          # 系统标准名: Revenue_Total
    raw_name: str               # 财报原始名: 营业总收入
    value: float                # 统一换算为元后的数值
    original_unit: str          # 原始单位: 万元
    report_type: str            # 合并/母公司

class FinancialStatement(BaseModel):
    """一份完整的财务数据, 包含三大报表"""
    company_name: str
    stock_code: str
    year: int
    report_type: str
    balance_sheet: dict[str, FinancialField] # 资产负债表
    income_statement: dict[str, FinancialField] # 利润表
    cashflow: dict[str, FinancialField] # 现金流量表
    validation: dict # 勾稽校验结果
```

schemas/tags.py — 标签结构

```
from pydantic import BaseModel

class HardTag(BaseModel):
    """硬标签: 行业分类"""
    system: str          # 分类体系: 同花顺三级行业
    value: str           # 具体值: 白酒

class SoftTag(BaseModel):
    """软标签: 商业模式特征"""
    dimension: str        # 维度: 资产结构
    value: str           # 值: 重资产

class CompanyTags(BaseModel):
    """一家公司的完整标签"""
    company_name: str
    stock_code: str
    hard_tags: list[HardTag]
    soft_tags: list[SoftTag]
```

schemas/anomaly.py — 异常数据

```
from pydantic import BaseModel

class LogicAnomaly(BaseModel):
    """B+层: 逻辑异常检查结果"""
    check_name: str          # 检查项: 净现比
    value: float             # 实际值
    threshold: float         # 阈值
    severity: float          # 严重度评分
    summary: str             # 异常描述

class DeviationAnomaly(BaseModel):
    """C层: MAD 偏差异常"""
    indicator: str           # 指标: 存货周转率
    actual_value: float       # 实际值
    benchmark_value: float   # 同行中位数
    mad_multiple: float      # 偏离 MAD 倍数
    severity: str            # normal / abnormal / extreme
```

schemas/benchmark.py — 基准数据

```
from pydantic import BaseModel

class PeerCompany(BaseModel):
    """匹配到的同行公司"""
    name: str
    stock_code: str
    similarity_score: float  # 软标签匹配相似度
    financials: dict         # 关键财务指标

class Benchmark(BaseModel):
    """横向 + 纵向基准"""
    peer_median: dict[str, float]  # 同行中位数
    historical_mean: dict[str, float]  # 自身5年均值
    peer_companies: list[PeerCompany]
```

schemas/reasoning.py — 推理结果

```
from pydantic import BaseModel

class Hypothesis(BaseModel):
    """D1路2: 一条推演假设"""
    hypothesis: str          # 假设内容
    reasoning: str           # 推演依据
    source: str              # 依据来源: 宏观事实 / 标签 / 偏差

class Explanation(BaseModel):
    """D1路1: 从原文提取的解释"""
    summary: str             # 解释摘要
    source_text: str         # 原文引用
    page_number: int         # 年报页码
    is_vague: bool          # 是否含糊其辞

class ProbabilityAssignment(BaseModel):
    """D2层: 一个异常的概率分配结果"""
    anomaly_indicator: str    # 异常指标
    anomaly_source: str       # B+ / C
    lookups: list[Explanation] # D1路1的结果
    hypotheses: list[Hypothesis] # D1路2的结果
    probabilities: dict[str, float] # 归因及其概率
    # 示例: {"经营滞销": 0.6, "战略备货": 0.3, "其他": 0.1}
```

schemas/report.py — 最终报告

```
from pydantic import BaseModel

class ScoreBreakdown(BaseModel):
    """E1层: 打分明细"""
    base_score: float          # 100
    c_layer_deduction: float    # C层扣分
    bplus_layer_deduction: float # B+层扣分
    final_score: float          # 最终得分

class Report(BaseModel):
    """E2层: 完整报告"""
    company_name: str
    stock_code: str
    report_year: int

    # 模块1
    overall_assessment: dict
    # { score: 85, tier: "中等置信度", peer_comparison: [...] }

    # 模块2
    core_anomalies: list[ProbabilityAssignment]

    # 模块3
    bull_points: list[str]      # 看多支持点
    bear_points: list[str]      # 看空风险点
```

pipeline/ — 全流程编排

```
pipeline/
├── __init__.py
├── orchestrator.py          # 主调度器
├── context.py               # 层间数据传递上下文
└── step_registry.py         # 步骤注册表
```

pipeline/orchestrator.py — 主调度器

用途：流水线引擎。按顺序调度 $0 \rightarrow A \rightarrow B \rightarrow B+ \rightarrow C \rightarrow D \rightarrow E$ 各层，处理层间数据传递和错误传播。

```

# 核心逻辑示意
class Orchestrator:
    def run(self, pdf_path: str) -> Report:
        ctx = PipelineContext()

        # 第0层: PDF提取
        ctx.raw_doc = layer_0_extract(pdf_path)

        # A层与B层理论上可并行, 但为了清晰按序执行
        self._run_layer_a(ctx)    # A0→A1→A2
        self._run_layer_b(ctx)    # B0→B1

        # B+层: 前置检查
        ctx.logic_anomalies = run_bplus(ctx.financials)

        # C层: 偏差计算
        ctx.deviation = run_c(ctx.financials, ctx.benchmark)

        # D层: 推理
        ctx.reasoning = run_d(ctx)

        # E层: 打分输出
        report = run_e(ctx)

    return report

```

边界约定:

- Orchestrator 不包含任何业务逻辑, 只负责"把数据从上一层输出传给下一层输入"
- 每个步骤可以独立运行和独立测试

pipeline/context.py — 层间上下文

用途: 一个贯穿全流程的"数据背包", 每一层的输出都往里放, 后一层从里面取。

```

from dataclasses import dataclass

@dataclass
class PipelineContext:
    raw_doc: Optional[RawDocument] = None
    # A层
    macro_facts: Optional[list[str]] = None
    tags: Optional[CompanyTags] = None
    benchmark: Optional[Benchmark] = None
    # B层
    financials: Optional[FinancialStatement] = None
    parent_financials: Optional[FinancialStatement] = None
    validation_passed: bool = False
    # B+层
    logic_anomalies: Optional[list[LogicAnomaly]] = None
    # C层
    deviations: Optional[list[DeviationAnomaly]] = None
    # D层
    reasoning_results: Optional[list[ProbabilityAssignment]] = None
    # E层
    score: Optional[ScoreBreakdown] = None
    report: Optional[Report] = None

```

pipeline/step_registry.py — 步骤注册表

用途：把各层的入口函数注册到统一的"步骤映射表"，使得 orchestrator 可以不直接 import 各层模块，而是通过 registry 查找。

```

STEP_REGISTRY: dict[str, Callable] = {}

def register(name: str):
    """装饰器：注册一个步骤"""
    def decorator(func):
        STEP_REGISTRY[name] = func
        return func
    return decorator

```

设计理由：

- 每层自己暴露入口，用 `@register("layer_b1")` 装饰，orchestrator 不需要知道具体实现
 - 支持"跳过某一步调试"： `orchestrator.run(skip=["layer_c"])`
 - 未来支持替换实现（如更换 PDF 解析器）只需注册新函数
-

layers/ — 六层功能实现

```
layers/
├── __init__.py
├── layer_0_extract/
├── layer_a_benchmark/
├── layer_b_extract/
├── layer_bplus_internal/
├── layer_c_deviation/
├── layer_d_reasoning/
└── layer_e_output/
```

层间约定的通用规则：

- 每一层通过 `step_registry.register(name)` 暴露一个入口函数
- 入口函数签名统一为 `def run(ctx: PipelineContext) -> Any`
- 纯代码模块（B1、B+、C、E1）不导入任何 LLM 相关模块
- LLM 模块（A0、A1、B0、D1、D2）统一通过 `llm/client.py` 调用

layers/layer_0_extract/ — PDF → JSON

```
layer_0_extract/
├── __init__.py
├── pdf_parser.py           # MinerU / pdfplumber 调用
├── chunker.py             # 按四大区块切分
└── merger.py              # 处理跨页表格拼接
```

文件	用途	由谁负责	是否涉及LLM
<code>__init__.py</code>	注册步骤 <code>layer_0</code> ，暴露 <code>run(pdf_path) -> RawDocument</code>	-	×
<code>pdf_parser.py</code>	调用外部 PDF 解析工具（MinerU）， 提取原始文字和表格数据	PDF 解析	×
<code>chunker.py</code>	根据章节标题规则，把提取结果切分为四大区块： 财务数据、管理层讨论、附注、公司基本情况	PDF 解析	×
<code>merger.py</code>	处理跨页表格的拼接和去重	PDF 解析	×

设计说明：

- 第 0 层是纯工具链，不涉及任何业务逻辑
- 输出格式由 `schemas/raw_doc.py` 严格定义
- 后期可以替换 PDF 解析工具（如从 MinerU 换成 PyMuPDF），只需改 `pdf_parser.py`

layers/layer_a_benchmark/ — 行业定性 & 找基准

```
layer_a_benchmark/
├── __init__.py
├── a0_macro_search.py      # LLM: 联网搜研报
├── a1_tagging.py          # LLM: 打行业标签
└── a2_matcher.py          # 代码: 匹配相似企业 + 算基准
```

文件	用途	由谁负责	是否涉及LLM
__init__.py	注册步骤 layer_a ，按序调用 A0→A1→A2	编排层	✗（编排）
a0_macro_search.py	加载 config/prompts/a0_macro_search.yaml ，调用 LLM 联网搜索，解析结果中的客观事实列表	LLM 应用	✓
a1_tagging.py	加载 config/prompts/a1_tagging.yaml ，调用 LLM 给公司打硬标签和软标签，返回 CompanyTags	LLM 应用	✓
a2_matcher.py	纯代码逻辑：① 硬标签锁定行业 ② 软标签算余弦相似度 ③ 取 3~5 家最相似同行 ④ 计算同行中位数 + 自身历史均值 → 输出 Benchmark	数据分析	✗

设计说明：

- A2 是纯代码——"大模型打完标签后，匹配和计算不能让 LLM 掺和"
- 同行数据库当前可以是 CSV/Parquet 文件放在 data/benchmarks/ ，未来可扩展为 SQLite 或向量数据库

layers/layer_b_extract/ — 财务数据提取

```
layer_b_extract/
├── __init__.py
├── b0_semantic_guide.py    # LLM: 读表头做字段映射
├── b1_extractors.py       # 代码: 定位表格、取数
└── b1_validators.py      # 代码: 勾稽关系校验
```

文件	用途	由谁负责	是否涉及LLM
__init__.py	注册步骤 layer_b ，按序调用 B0→B1	编排层	✗
b0_semantic_guide.py	大模型读报表表头，输出：① 字段名映射 ("营业总收入"→ Revenue_Total) ② 单位 ("万元") ③ 报表类型（合并/母公司）	LLM 应用	✓

文件	用途	由谁负责	是否涉及LLM
b1_extractors.py	纯 Pandas 代码：根据 B0 的指引，精确定位表格的行列坐标，提取数字，统一换算为"元"	数据处理	✗
b1_validators.py	纯代码：做勾稽关系校验（资产=负债+权益、未分配利润变动等）→ 返回 FinancialStatement.validation	数据处理	✗

设计说明：

- **B0（LLM 读表头）**与 **B1（代码取数）** 严格分离，是"隔离原则"的集中体现
- B0 只输出"指引"（字段 A 在表格第 3 行第 2 列），不接触任何数值
- B1 只做"机械操作"（取第 3 行第 2 列的值乘 10000），不做任何语义判断
- b1_validators.py 中的校验失败会导致流水线**阻断终止**（直接跳到 E2 输出异常报告）

layers/layer_bplus_internal/ — 造假初筛

```
layer_bplus_internal/  
├── __init__.py  
└── logic_checks.py          # 净现比 / 存贷双高 / 母子分离
```

文件	用途	由谁负责	是否涉及LLM
__init__.py	注册步骤 layer_bplus ，调用所有检查项	编排层	✗
logic_checks.py	实现三项检查，每项输出 LogicAnomaly ；同时提供 calc_severity() 函数给 E1 层复用	财务分析	✗

函数签名：

```
def check_net_profit_cash_ratio(financials: FinancialStatement) -> Optional[LogicAnomaly]  
    """净现比/收现比检查，返回 None 表示正常"""  
  
def check_deposit_loan_dual_high(financials: FinancialStatement) -> Optional[LogicAnomaly]  
    """存贷双高检查"""  
  
def check_subsidiary_fund_separation(  
    consolidated: FinancialStatement,  
    parent: FinancialStatement  
) -> Optional[LogicAnomaly]:  
    """母子资金分离度检查"""  
  
def calc_severity(check_name: str, value: float, total_assets: float) -> float:  
    """根据具体的逻辑异常类型和数值，计算严重度修正分（供 E1 层调用）"""
```

layers/layer_c_deviation/ — MAD 偏差计算

```
layer_c_deviation/
├── __init__.py
├── mad_calculator.py          # MAD 核心算法
└── deviation_scorer.py        # 异常指标筛选与排序
```

文件	用途	由谁负责	是否涉及LLM
__init__.py	注册步骤 layer_c	编排层	✗
mad_calculator.py	实现 MAD 算法： 对每个指标计算实际值偏离同行中位数的 MAD 倍数	数学/ 统计	✗
deviation_scorer.py	根据 thresholds.yaml 的阈值，筛选出超过 mad_threshold 的指标，标记严重等级	数学/ 统计	✗

核心函数：

```
def calc_mad(values: list[float]) -> float:
    """计算一组数据的中位数绝对偏差"""
    median = np.median(values)
    return np.median(np.abs(values - median))

def calc_deviation_multiple(
    actual_value: float,
    peer_values: list[float]
) -> tuple[float, float, float]:
    """
    返回: (peer_median, mad, deviation_multiple)
    deviation_multiple = |actual - median| / mad
    若 mad = 0, 返回 inf
    """
```

layers/layer_d_reasoning/ — 原因查找 + 概率分配

```
layer_d_reasoning/
├── __init__.py
├── d1_dual_path.py           # 双路径推理编排
├── d1_lookup_notes.py        # 路1: 查年报原文
├── d1_hypothesis.py          # 路2: 推演假设
└── d2_probability.py         # 概率分配
```

文件	用途	由谁负责	是否涉及LLM
<code>__init__.py</code>	注册步骤 <code>layer_d</code> ，接收 B+ 和 C 的异常清单	编排层	✗
<code>d1_dual_path.py</code>	调度器：遍历每个异常，同时调用路1和路2，收集结果	编排	✗（编排）
<code>d1_lookup_notes.py</code>	路1：从年报原文的【附注】和【管理层讨论】中搜索相关解释，调用 LLM 提取关键信息	LLM 应用	✓
<code>d1_hypothesis.py</code>	路2：不看原文，用 A0 宏观事实 + A1 标签 + 偏差数据，调用 LLM 发散假设	LLM 应用	✓
<code>d2_probability.py</code>	结合两路结果，调用 LLM 分配概率，输出 <code>ProbabilityAssignment</code>	LLM 应用	✓

LLM 调用密集区域：

- D 层是系统中最依赖 LLM 的层次，每个异常都会调用 2~3 次 LLM（路1 + 路2 + 概率分配）
- 如果报告有 10 个异常，D 层可能调用 20~30 次 LLM —— 这是主要的成本中心

layers/layer_e_output/ — 最终打分 & 报告

```
layer_e_output/  
├── __init__.py  
├── e1_scoring.py           # 代码：置信度打分  
└── e2_report_gen.py       # 报告组装（含少量 LLM 文本润色）
```

文件	用途	由谁负责	是否涉及LLM
<code>__init__.py</code>	注册步骤 <code>layer_e</code> ，调用 E1→E2	编排层	✗
<code>e1_scoring.py</code>	纯代码：根据 <code>weights.yaml</code> + 概率结果 + MAD 偏离度 + 严重度修正，算最终得分	数学/ 评分	✗
<code>e2_report_gen.py</code>	组装结构化报告：E1 得分 + D 层概率归因 + B+ 逻辑检查 + A2 同行对比。其中【多空逻辑栈】的文本描述部分可由 LLM 润色	数据组装	✓ (仅文本润色)

E1 打分公式在代码中的表达：

```

def calc_score(
    weights: dict,
    deviations: list[DeviationAnomaly],
    probabilities: list[ProbabilityAssignment],
    logic_anomalies: list[LogicAnomaly],
) -> ScoreBreakdown:
    base = 100.0

    # C层扣分
    c_deduction = 0.0
    for dev, prob in zip(deviations, probabilities):
        # 基础影响权重 × (1 + MAD偏离度/3) = 实际影响权重
        base_weight = weights[dev.indicator].base_weight
        effective_weight = base_weight * (1 + dev.mad_multiple / 3)

        # 对每个归因算期望影响
        for cause, p in prob.probabilities.items():
            impact = p * effective_weight

        # C层扣分 = 期望影响 × MAD偏离度
        c_deduction += impact * dev.mad_multiple

    # B+层扣分 = 权重 × 修正系数
    bplus_deduction = sum(
        -3 * anomaly.severity # 造假权重-3 × 严重度
        for anomaly in logic_anomalies
    )

    final = max(0.0, base - c_deduction - bplus_deduction)
    return ScoreBreakdown(base, c_deduction, bplus_deduction, final)

```

llm/ — LLM 调用统一网关

```

llm/
├── __init__.py
├── client.py           # 统一的 LLM 调用接口
├── prompt_loader.py   # 加载和渲染 prompt 模板
└── response_parser.py # 解析 LLM 输出

```

llm/client.py — LLM 客户端

用途：屏蔽不同 LLM 厂商的 API 差异，提供统一调用接口。

```

class LLMClient:
    def __init__(self, provider: str = "anthropic"):
        # 根据 provider 初始化对应 client
        pass

    def chat(
        self,
        prompt_name: str,          # 引用 config/prompts/ 下的模板
        variables: dict,           # 模板变量
        response_schema: type,     # 期望的 Pydantic 输出结构（可选）
    ) -> str | BaseModel:
        """
        1. prompt_loader 加载 prompt_name 对应的模板
        2. 渲染模板（填充 variables）
        3. 调用 LLM API（支持 streaming 和结构化输出）
        4. response_parser 解析输出
        5. 如果提供了 response_schema，用 Pydantic 校验
        """

```

支持的特性：

- 多厂商切换（Anthropic / OpenAI / 本地模型）
- Token 计数与用量日志
- 自动重试（指数退避）
- 结构化输出（JSON mode / tool use）
- 超时控制

llm/prompt_loader.py — Prompt 加载器

```

class PromptLoader:
    def __init__(self, prompts_dir: str = "config/prompts"):
        self.prompts_dir = prompts_dir
        self._cache: dict[str, dict] = {}

    def load(self, name: str) -> dict:
        """按 name 加载 prompt，支持缓存"""
        pass

    def render(self, name: str, variables: dict) -> str:
        """加载并渲染 prompt 模板"""
        pass

```

llm/response_parser.py — 输出解析器

```
class ResponseParser:
    @staticmethod
    def parse_json(raw: str) -> dict:
        """解析 LLM 返回的 JSON 字符串，处理常见格式问题"""
        pass

    @staticmethod
    def extract_facts(raw: str) -> list[str]:
        """从 LLM 返回的文本中提取客观事实条目"""
        pass

    @staticmethod
    def validate_against_schema(
        data: dict,
        schema: type[BaseModel]
    ) -> BaseModel:
        """用 Pydantic 校验并转换"""
        pass
```

data/ — 数据目录

```
data/
├── raw/                # 原始 PDF
├── processed/          # 第0层输出的 JSON
├── benchmarks/         # 行业基准数据库
└── outputs/            # 最终报告
```

data/raw/ — 原始 PDF

用途：存放待分析的年报 PDF 文件，按 `{stock_code}_{year}.pdf` 命名。

约定：

- 文件较大（一般 5~50MB），通过 `.gitignore` 排除
- 通过 `.env` 中的 `DATA_DIR` 自定义路径
- 初次使用需手动放入或通过脚本下载

组织结构：

```
raw/
├── 600519_2024.pdf    # 贵州茅台 2024 年年报
├── 600519_2023.pdf
├── 000858_2024.pdf    # 五粮液 2024 年年报
└── ...
```

data/processed/ — 处理后的 JSON

用途：第 0 层输出的切片结果，格式为 `schemas/raw_doc.py` 定义的 JSON。

约定：

- 和 raw 一样不进 git
- 如果 `processed/` 下已有对应 JSON，第 0 层可以跳过（加速调试）
- 按 `{stock_code}_{year}.json` 命名

data/benchmarks/ — 行业基准数据库

用途：A2 层使用的财务指标基准数据。

最初形态：手动整理的 CSV 文件

```
stock_code,industry,year,indicator,value
600519,白酒,2024,存货周转率,0.18
000858,白酒,2024,存货周转率,0.32
...
```

演进路径：

1. 初期：手动 CSV + 人工维护
2. 中期：脚本定期从数据源更新
3. 远期：SQLite / 时序数据库

约定：

- 基准数据是从公开数据源整理的客观数据，不涉及分析结论
- CSV 文件可以进 git（样本数据），完整数据库不进

data/outputs/ — 最终报告

用途：E2 层生成的结构化报告，JSON 格式。

```
{
  "company": "贵州茅台",
  "year": 2024,
  "score": 92.5,
  "anomalies": [],
  "report_generated_at": "2026-07-26T14:30:00Z"
}
```

约定：

- 不进 git
- 按 `{stock_code}_{year}_report.json` 命名
- 每次分析覆盖旧文件

tests/ — 测试

```
tests/
├── __init__.py
├── conftest.py           # pytest 共享 fixtures
├── test_layers/         # 各层单元测试
│   ├── test_b1_validators.py
│   ├── test_c_mad.py
│   ├── test_e1_scoring.py
│   └── test_a2_matcher.py
├── test_llm/            # LLM 网关测试
│   ├── test_prompt_loader.py
│   └── test_response_parser.py
├── test_pipeline/       # 集成测试
│   ├── test_orchestrator.py
│   └── test_full_pipeline.py
└── fixtures/           # 测试用 mock 数据
    ├── sample_financials.json
    ├── mock_benchmarks.csv
    └── sample_raw_doc.json
```

tests/conftest.py — 共享 Fixtures

用途：定义在多个测试文件中复用的 fixture，如 mock 财务数据、mock LLM 响应。

```
@pytest.fixture
def sample_financials() -> FinancialStatement:
    """返回一份干净的测试用财务数据"""
    with open("tests/fixtures/sample_financials.json") as f:
        return FinancialStatement.model_validate_json(f.read())

@pytest.fixture
def mock_llm_client(monkeypatch):
    """mock LLM 客户端，返回预设响应，不实际调用 API"""
    def mock_chat(*args, **kwargs):
        return {"result": "mock_response"}
    monkeypatch.setattr("llm.client.LLMClient.chat", mock_chat)
```

tests/test_layers/ — 各层单元测试

全部是纯逻辑测试，不需要 LLM 参与（mock 掉 LLM 调用）。

test_b1_validators.py 示例：

```
def test_balance_sheet_equation():
    """资产总计 = 负债合计 + 所有者权益合计"""
    data = load_sample_financials()
    result = validate_balance_sheet(data)
    assert result.is_valid

def test_detects_imbalanced_sheet():
    """故意弄错一个数，校验应该失败"""
    ...

def test_unit_conversion():
    """万元 → 元的换算是否正确"""
    ...
```

test_c_mad.py 示例：

```
def test_mad_normal_case():
    values = [10, 12, 11, 13, 100] # 100 是异常值
    mad = calc_mad(values)
    assert mad < 2.0 # MAD 对极端值稳健

def test_deviation_detection():
    """大幅偏离中位数的指标应被标记为 extreme"""
    ...
```

test_e1_scoring.py 示例：

```
def test_base_score():
    """无异常时应保持 100 分"""
    score = calc_score(
        weights=default_weights,
        deviations=[],
        probabilities=[],
        logic_anomalies=[],
    )
    assert score.final_score == 100.0

def test_deduction_with_anomaly():
    """有异常时应适当扣分"""
    ...
```

tests/test_llm/ — LLM 网关测试

```
# 不实际调用 LLM API，只测试 prompt 加载和输出解析的逻辑
def test_prompt_loading():
    loader = PromptLoader("config/prompts")
    prompt = loader.load("a0_macro_search")
    assert prompt["name"] == "a0_macro_search"

def test_response_json_parser():
    parser = ResponseParser()
    raw = '{"key": "value"}' # LLM 返回的 JSON
    result = parser.parse_json(raw)
    assert result["key"] == "value"

def test_response_json_with_markdown_fencing():
    """LLM 有时会用 ```json``` 包裹输出，解析器要能处理"""
    raw = ```json\n{"key": "value"}\n```
    result = parser.parse_json(raw)
    assert result["key"] == "value"
```

tests/test_pipeline/ — 集成测试

```
def test_full_pipeline_with_mock():
    """使用 mock 数据跑完整流水线，验证流程不报错且输出格式正确"""
    ctx = PipelineContext()
    ctx.raw_doc = load_fixture("sample_raw_doc.json")

    orchestrator = Orchestrator()
    report = orchestrator.run(ctx)

    assert isinstance(report, Report)
    assert 0 <= report.score.final_score <= 100
```

notebooks/ — 探索性分析

```
notebooks/
├─ prototype_deviation.ipynb # MAD 算法原型
├─ explore_weight_formula.ipynb # 评分公式探索
└─ sample_analysis.ipynb # 一份真实年报的演示分析
```

用途：

- 开发和验证阶段的"实验场"
- 重要算法（MAD、评分公式）先在这里跑通，再迁移到正式代码

- 作为演示和报告使用

约定：

- Notebook 中的数据路径使用相对于仓库根的路径
- 正式代码开发完后，notebook 可以被删除或存档
- Notebook 不带敏感数据（或使用 mock 数据）

scripts/ — 运维/工具脚本

```
scripts/
├─ run_pipeline.py           # 命令行入口
├─ build_benchmark_db.py    # 初始化行业基准数据库
├─ validate_pdf.py         # 快速验证 PDF 可解析性
├─ batch_analyze.py        # 批量分析多个公司
└─ show_report.py          # 在终端格式化显示报告
```

scripts/run_pipeline.py — 主入口

用途：用户运行系统的唯一命令行入口。

```
# 基本用法：分析一份 PDF
python scripts/run_pipeline.py --pdf data/raw/600519_2024.pdf

# 输出指定目录
python scripts/run_pipeline.py --pdf xxx.pdf --output ./my_reports

# 调试模式：从某一步开始
python scripts/run_pipeline.py --pdf xxx.pdf --from layer_c

# 跳过 LLM 调用（只用缓存）
python scripts/run_pipeline.py --pdf xxx.pdf --no-llm
```

scripts/build_benchmark_db.py — 基准数据初始化

用途：从公开数据源拉取行业财务指标数据，构建 `data/benchmarks/` 数据库。

```
# 初始化白酒行业基准
python scripts/build_benchmark_db.py --industry 白酒

# 从本地 CSV 导入
python scripts/build_benchmark_db.py --csv ./my_data.csv
```

scripts/validate_pdf.py — PDF 验证工具

用途：在运行完整流水线前检查 PDF 文件是否可解析。

```
python scripts/validate_pdf.py data/raw/600519_2024.pdf
# 输出:
# ✔ PDF 可读取 (256页)
# ✔ 包含财务表格
# ✔ 包含管理层讨论章节
# ✖ 附注部分不完整
```

scripts/batch_analyze.py — 批量分析

用途：批量分析多份年报。

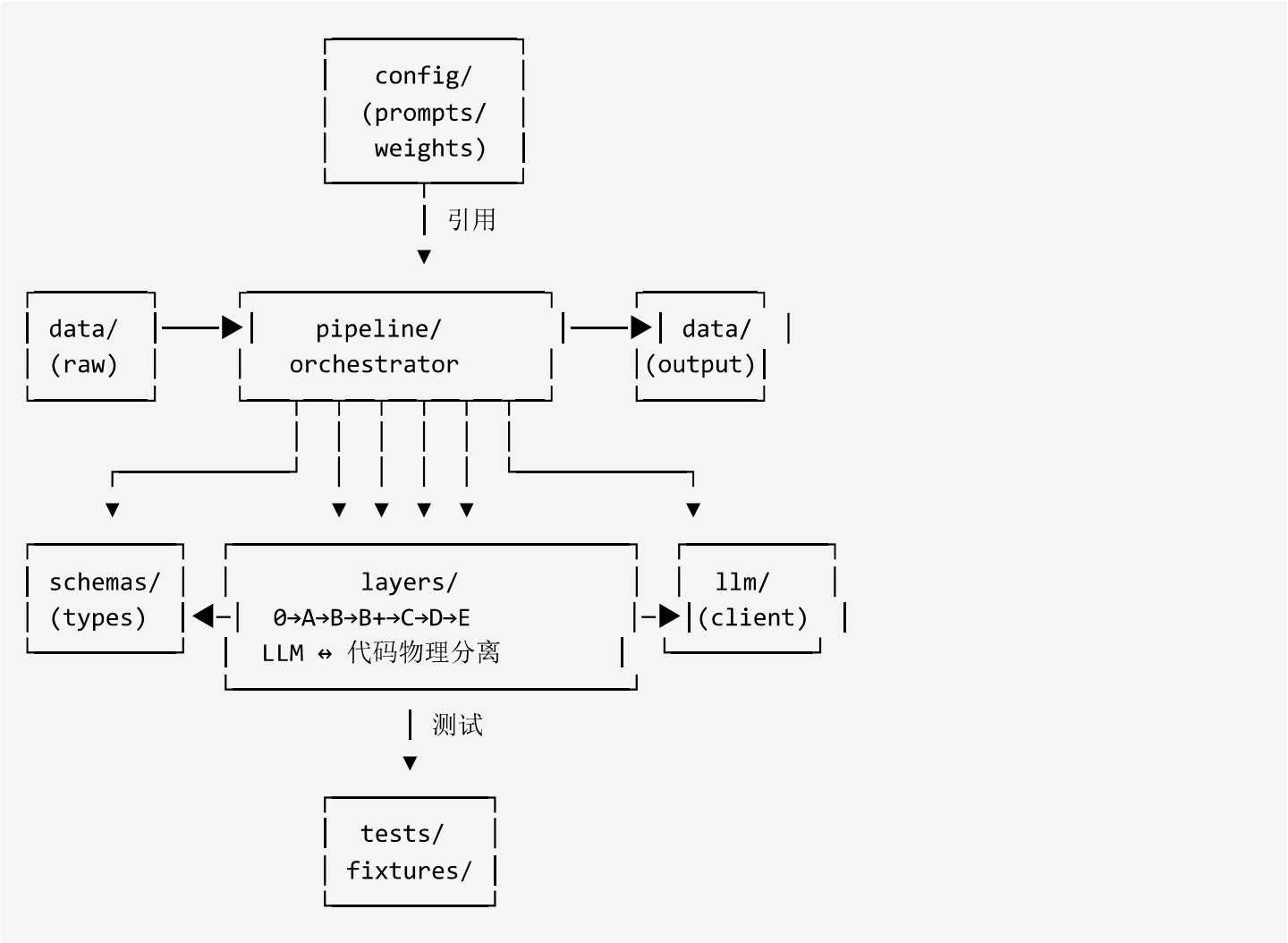
```
python scripts/batch_analyze.py --pdfs data/raw/ --output data/outputs/
```

scripts/show_report.py — 报告查看器

用途：在终端格式化显示已生成的报告。

```
python scripts/show_report.py data/outputs/600519_2024_report.json
# 终端输出格式化的报告摘要
```

附录：目录依赖关系图



关键关系：

- `schemas/` 被每一层引用：定义数据格式
- `config/` 被 `layers/` 和 `llm/` 引用：提供配置参数
- `llm/` 被 `layers/` 引用：提供统一的 LLM 调用能力
- `pipeline/` 引用 `layers/`、`schemas/`、`llm/`、`config/`：编排一切
- `tests/` 引用 `layers/`、`llm/`、`pipeline/`：测试依赖
- `scripts/` 引用 `pipeline/`：命令行入口
- `data/` 和 `notebooks/` 独立，不参与依赖

本文档应与 [架构-v4.1.md](#) 配合阅读。